

PROJET DE FIN D'ETUDES 2025/2026

CACHE DISTRIBUE AVEC REDIS

Projet 11

Omar Sarsar & Iliass Hariz

CONTEXTE ET OBJECTIFS



Marketplace SaaS

Application marketplace
SaaS avec des
ralentissements croissants
sur les pages produits



Performances

Ralentissements sur les
pages dues a la charge sur la
base de donnees
PostgreSQL



Cache Redis

Solution : integration d'un
cache distribue Redis pour
reduire les temps de reponse



Docker Compose

Deploiement containerise
avec Docker Compose pour
l'orchestration des services



Mesures

Mesure et comparaison des
performances avant et apres
l'integration du cache

Objectif Principal

Ameliorer les performances de l'application marketplace en implementant un **cache distribue Redis** qui verifie le cache avant d'interroger PostgreSQL. Cette approche permet de :

- Reduire le **temps de reponse** des pages produits
- Diminuer la **charge sur la base de donnees**
- Maintenir la **coherence des donnees** via une strategie d'invalidation

ARCHITECTURE DU SYSTEME

Pattern Cache-Aside

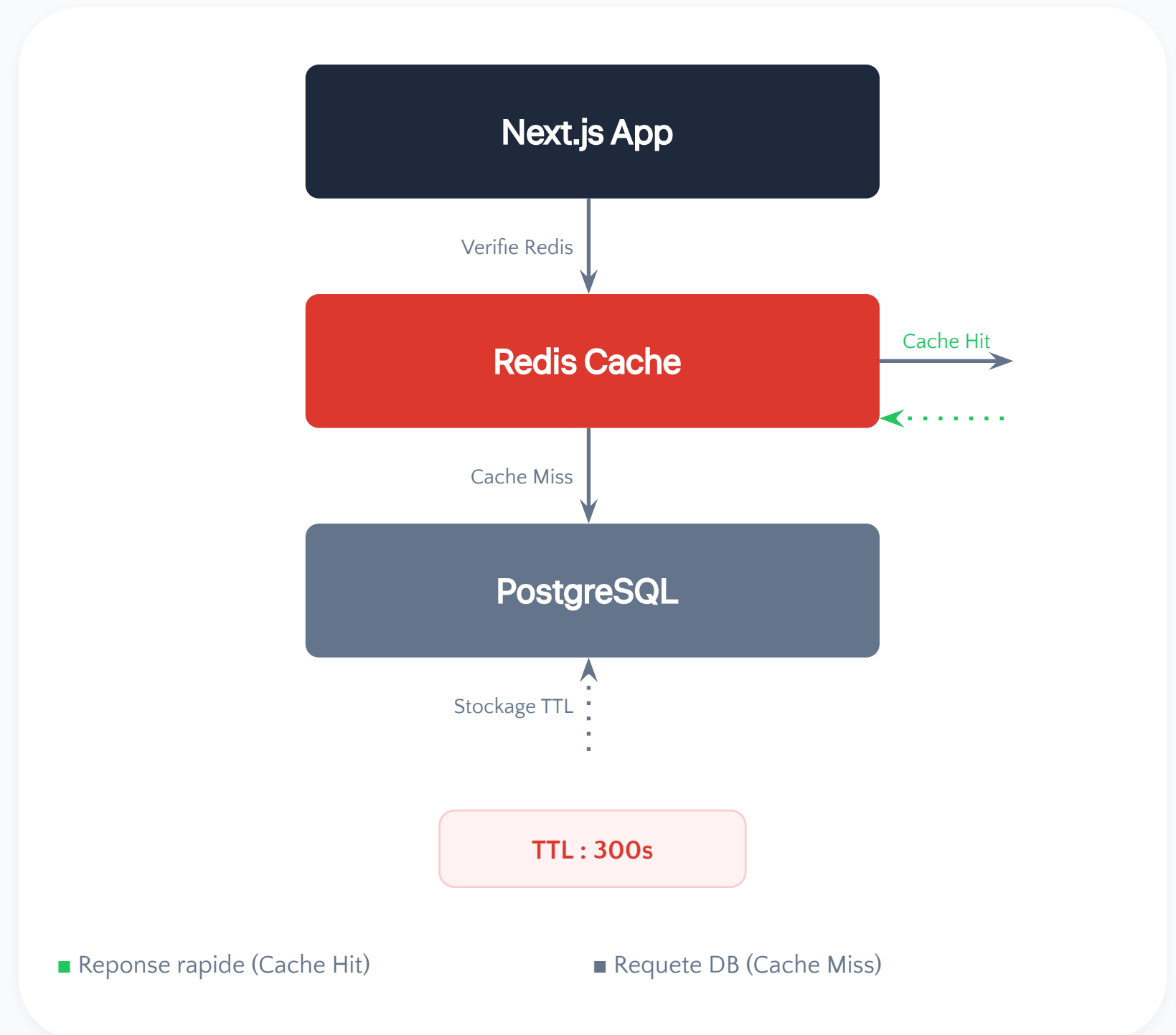
L'architecture suit le pattern **Cache-Aside** (Lazy Loading) :

Verification **Redis** avant la base de donnees

Stockage avec **TTL configurable**

Reduction significative du **temps de reponse**

Le cache est mis a jour uniquement en cas de "cache miss", minimisant ainsi les acces a PostgreSQL.



INFRASTRUCTURE DOCKER

Service	Image	Port	Role
PostgreSQL	postgres:16-alpine	5434:5432	Base de donnees
Redis	redis:7-alpine	6379:6379	Cache
Next.js	Dockerfile.dev	3000:3000	Application

Caracteristiques Cles



Orchestration

Docker Compose pour la gestion et l'orchestration de tous les services



Volume Persistant

Volume redis_data pour la persistance des donnees du cache



Healthcheck

Healthcheck integre pour Redis assurant la disponibilite du service



Connexion

Variable d'environnement REDIS_URL pour la connexion a Redis

COUCHE D'ABSTRACTION



src/lib/cache.ts

Interface simple et réutilisable pour Redis



cacheGet()

Récupération des données depuis Redis avec **typage**

TypeScript.

Parse automatiquement les données JSON stockées et les retourne avec le type générique spécifié.



cacheSet()

Stockage des données avec **TTL configurable.**

Séréalise les données en JSON et définit une expiration automatique. TTL par défaut : 300 secondes.



TTL par Défaut

300 secondes (5 minutes)

INTEGRATION DANS LES ACTIONS

1

Generation Cle Unique

Creation d'une cle basee sur le nom de la fonction et ses parametres

2

Verification Redis

Interrogation du cache avec `cacheGet()` pour verifier l'existence des donnees

3

Cache Miss / Interroger DB

Si absent du cache, interrogation de PostgreSQL pour recuperer les donnees

4

Stockage TTL

Stockage des resultats dans Redis avec `cacheSet()` et expiration automatique

5

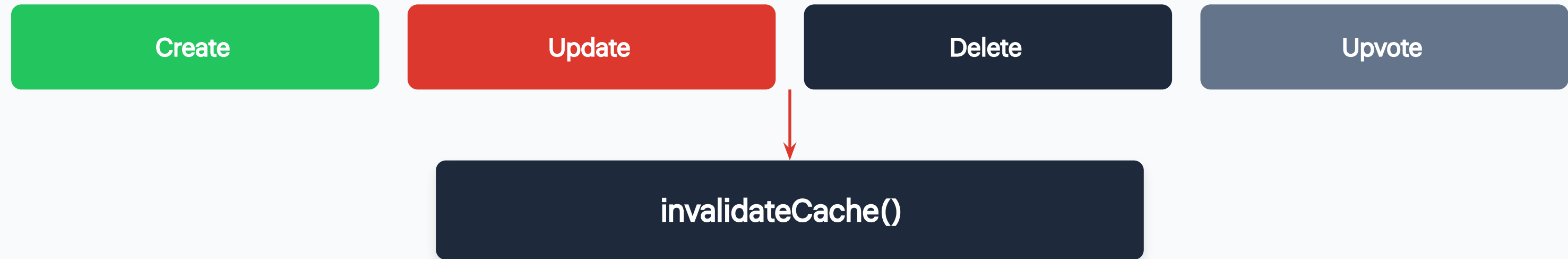
Application Lectures

Retour des donnees a l'application pour toutes les lectures suivantes



Ce flux garantit que les donnees sont toujours servies depuis le cache lorsque possible, reduisant drastiquement les acces a la base de donnees tout en maintenant des donnees fraiches grace au mecanisme de TTL.

INVALIDATION DU CACHE



Details de la Strategie



Invalidation Immediate

Chaque operation d'ecriture declenche immediatement l'invalidation du cache correspondant



Suppression des Cles

La fonction supprime les cles correspondantes de Redis pour forcer le rafraichissement



Expiration Automatique (TTL)

Le TTL sert de mecanisme d'expiration automatique en cas d'oubli d'invalidation



Coherence des Donnees

Objectif : maintenir la coherence entre le cache Redis et la base de donnees PostgreSQL

TESTS ET VALIDATION



8/8

Tests Passes avec Succes

100%

Taux de reussite

Couverture des Tests

01 Instance Redis Locale

Tests Jest avec une instance Redis locale pour des tests unitaires rapides et isolés

02 Operations Set, Get, Invalidation

Validation complete des operations de stockage, recuperation et invalidation du cache

03 Cas Limites

Gestion robuste des cas limites : valeurs null, undefined et donnees expirees

04 Pipeline CI

Tests automatisés exécutés dans la pipeline CI pour garantir la qualité continue

PERFORMANCE

12ms

Temps moyen AVEC Cache



856ms

Temps moyen SANS Cache

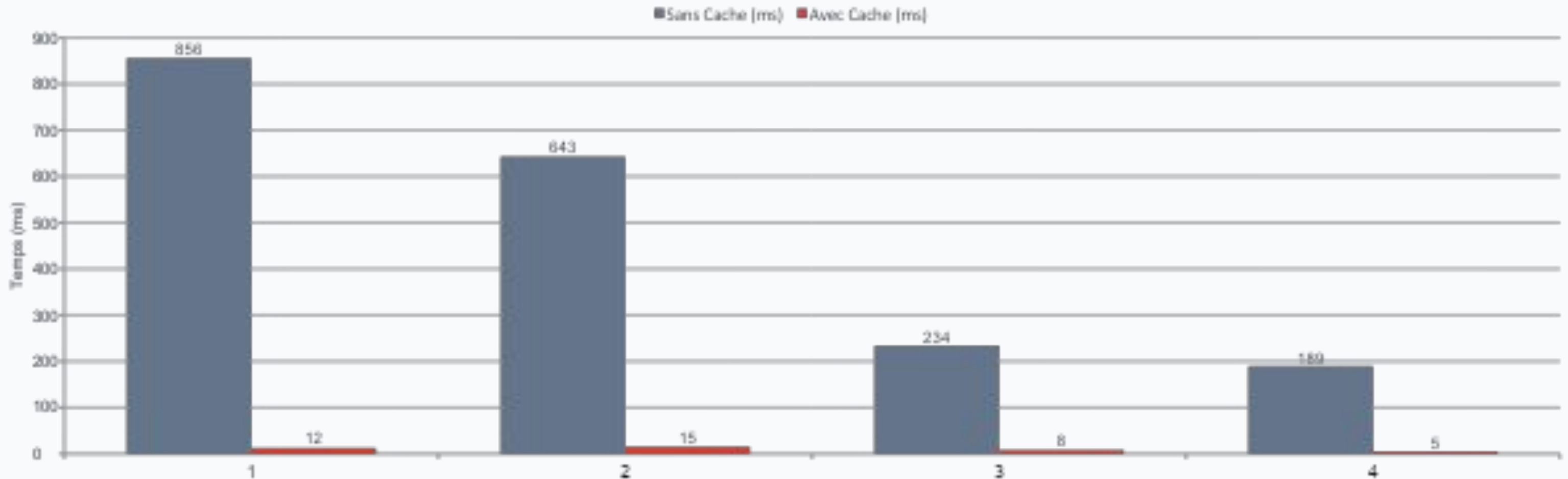


99%

Amelioration Globale



Comparaison par Endpoint



MERCI

Questions et Discussion

Omar Sarsar & Iliass Hariz

2026